Chapter 8: Instruction Encoding and ISA (Live)

Mohamed M. Sabry Aly

N4-02c-92

Learning Objectives

- Describe the instruction format of ARM instruction-set architecture (ISA)

- Describe differences between fixed and variable-length ISA

Overall Instruction Format

- Arithmetic & Logic instructions
  - ADD, ADC, SUB, SBC, RSB, AND, EOR, RSC, ORR, BIC

XXXCCS Rd, Rs1, Rs2 , {shft #}

31 28		24 21		19 16	15 12	11 7	6 5		3 0
Condition	000	Inst. type	S	Rs1	Rd	Shift size	shft	0	Rs2

XXXCCS Rd, Rs1, Rs2 , {shft Rshift}

31 28		24 21		19 16	15 12	11	7	6 5		3 0
Condition	000	Inst. type	S	Rs1	Rd	Rshift	0	shft	1	Rs2

XXXCCS Rd, Rs1, #immediate (rotated)

31 28		24 21		19 16	15 12	11 8	7 0
Condition	001	Inst. type	S	Rs1	Rd	#rot	8-bit immediate

Overall Instruction Format

- Arithmetic & Logic instructions
  - ADD, ADC, SUB, SBC, RSB, AND, EOR, RSC, ORR, BIC

XXXCCS Rd, Rs1, Rs2 , {shft #}

31 28		24 21		19 16	15 12	11 7	6 5		3 0
Condition	000	Inst. type	S	Rs1	Rd	Shift size	shft	0	Rs2

ADD R0,R1,R2,LSL #5

31 28		24 21		19 16	15 12	11 7	6 5		3 0
1110	000	0100	0	0001	0000	00101	00	0	0010

ADD R0,R1,R2

31 28		24 21		19 16	15 12	11 7	6 5		3 0
1110	000	0100	0	0001	0000	00000	00	0	0010

Q1 which of the following hexadecimal encoding of ADD R2,R1,R3

1. 0xE0812003
2. 0xF1021003
3. 0XE0831200
4. 0x00012003

XXXCCS Rd, Rs1, Rs2 , {shft #}

31 28 24 21 19 16 15 12 11 7 6 5 3 0

Condition	000	Inst. type	S	Rs1	Rd	Shift size	shft	0	Rs2

XXXCCS Rd, Rs1, Rs2 , {shft Rshift}

31 28 24 21 19 16 15 12 11 7 6 5 3 0

Condition	000	Inst. type	S	Rs1	Rd	Rshift	0	shft	1	Rs2

XXXCCS Rd, Rs1, #immediate (rotated)

31 28 24 21 19 16 15 12 11 8 7 0

Condition	001	Inst. type	S	Rs1	Rd	#rot	8-bit immediate

Code	Condition	Flags	Meaning
0000	**EQ**	Z = 1	Equal
0001	**NE**	Z = 0	Not equal
0010	**CS or HS**	C = 1	Higher or same, unsigned
0011	**CC or LO**	C = 0	Lower, unsigned
0100	**MI**	N = 1	Negative
0101	**PL**	N = 0	Positive or zero
0110	**VS**	V = 1	Overflow
0111	**VC**	V = 0	No overflow
1000	**HI**	C = 1 and Z = 0	Higher, unsigned
1001	**LS**	C = 0 or Z = 1	Lower or same, unsigned
1010	**GE**	N = V	Greater than or equal, signed
1011	**LT**	N != V	Less than, signed
1100	**GT**	Z = 0 and N = V	Greater than, signed
1101	**LE**	Z = 1 and N != V	Less than or equal, signed
1110	**AL**		Always.
1111	**NV**		Reserved (unused)

Code	Instruction
0000	**AND**
0001	**EOR**
0010	**SUB**
0011	**RSB**
0100	**ADD**
0101	**ADC**
0110	**SBC**
0111	**RSC**

Code	Instruction
1000	**TST**
1001	**TEQ**
1010	**CMP**
1011	**CMN**
1100	**ORR**
1101	**MOV**
1110	**BIC**
1111	**MVN**

Q1 which of the following hexadecimal encoding of ADD R2,R1,R3

1. 0xE0812003

2. 0xF1021003

3. 0XE0831200

4. 0x00012003

XXXCCS Rd, Rs1, Rs2 , {shft #}

31–28	27–25	24–21	20	19–16	15–12	11–7	6–5	4	3–0
Condition	000	Inst. type	S	Rs1	Rd	Shift size	shft	0	Rs2

XXXCCS Rd, Rs1, Rs2 , {shft Rshift}

31–28	27–25	24–21	20	19–16	15–12	11–8	7	6–5	4	3–0
Condition	000	Inst. type	S	Rs1	Rd	Rshift	0	shft	1	Rs2

XXXCCS Rd, Rs1, #immediate (rotated)

31–28	27–25	24–21	20	19–16	15–12	11–8	7–0
Condition	001	Inst. type	S	Rs1	Rd	#rot	8-bit immediate

Code	Condition	Flags	Meaning
0000	EQ	Z = 1	Equal
0001	NE	Z = 0	Not equal
0010	CS or HS	C = 1	Higher or same, unsigned
0011	CC or LO	C = 0	Lower, unsigned
0100	MI	N = 1	Negative
0101	PL	N = 0	Positive or zero
0110	VS	V = 1	Overflow
0111	VC	V = 0	No overflow
1000	HI	C = 1 and Z = 0	Higher, unsigned
1001	LS	C = 0 or Z = 1	Lower or same, unsigned
1010	GE	N = V	Greater than or equal, signed
1011	LT	N != V	Less than, signed
1100	GT	Z = 0 and N = V	Greater than, signed
1101	LE	Z = 1 and N != V	Less than or equal, signed
1110	AL		Always.
1111	NV		Reserved (unused)

Code	Instruction
0000	AND
0001	EOR
0010	SUB
0011	RSB
0100	ADD
0101	ADC
0110	SBC
0111	RSC

Code	Instruction
1000	TST
1001	TEQ
1010	CMP
1011	CMN
1100	ORR
1101	MOV
1110	BIC
1111	MVN

Q2 which of the following hexadecimal encoding of RSBEQS R2,R1,#5

1. 0xF1012005
2. 0x02712005
3. 0X01321005
4. 0x01212005

XXXCCS Rd, Rs1, Rs2 , {shft #}

31–28	27–25	24–21	20	19–16	15–12	11–7	6–5	4	3–0
Condition	000	Inst. type	S	Rs1	Rd	Shift size	shft	0	Rs2

XXXCCS Rd, Rs1, Rs2 , {shft Rshift}

31–28	27–25	24–21	20	19–16	15–12	11–8	7	6–5	4	3–0
Condition	000	Inst. type	S	Rs1	Rd	Rshift	0	shft	1	Rs2

XXXCCS Rd, Rs1, #immediate (rotated)

31–28	27–25	24–21	20	19–16	15–12	11–8	7–0
Condition	001	Inst. type	S	Rs1	Rd	#rot	8-bit immediate

Code	Condition	Flags	Meaning
0000	EQ	Z = 1	Equal
0001	NE	Z = 0	Not equal
0010	CS or HS	C = 1	Higher or same, unsigned
0011	CC or LO	C = 0	Lower, unsigned
0100	MI	N = 1	Negative
0101	PL	N = 0	Positive or zero
0110	VS	V = 1	Overflow
0111	VC	V = 0	No overflow
1000	HI	C = 1 and Z = 0	Higher, unsigned
1001	LS	C = 0 or Z = 1	Lower or same, unsigned
1010	GE	N = V	Greater than or equal, signed
1011	LT	N != V	Less than, signed
1100	GT	Z = 0 and N = V	Greater than, signed
1101	LE	Z = 1 and N != V	Less than or equal, signed
1110	AL		Always.
1111	NV		Reserved (unused)

Code	Instruction
0000	AND
0001	EOR
0010	SUB
0011	RSB
0100	ADD
0101	ADC
0110	SBC
0111	RSC

Code	Instruction
1000	TST
1001	TEQ
1010	CMP
1011	CMN
1100	ORR
1101	MOV
1110	BIC
1111	MVN

Q2 which of the following hexadecimal encoding of RSBEQS R2,R1,#5

1. 0xF1012005
2. 0x02712005
3. 0X01321005
4. 0x01212005

XXXCCS Rd, Rs1, Rs2 , {shft #}

31 28 24 21 19 16 15 12 11 7 6 5 3 0

Condition	000	Inst. type	S	Rs1	Rd	Shift size	shft	0	Rs2

XXXCCS Rd, Rs1, Rs2 , {shft Rshift}

31 28 24 21 19 16 15 12 11 7 6 5 3 0

Condition	000	Inst. type	S	Rs1	Rd	Rshift	0	shft	1	Rs2

XXXCCS Rd, Rs1, #immediate (rotated)

31 28 24 21 19 16 15 12 11 8 7 0

Condition	001	Inst. type	S	Rs1	Rd	#rot	8-bit immediate

Code	Condition	Flags	Meaning
0000	EQ	Z = 1	Equal
0001	NE	Z = 0	Not equal
0010	CS or HS	C = 1	Higher or same, unsigned
0011	CC or LO	C = 0	Lower, unsigned
0100	MI	N = 1	Negative
0101	PL	N = 0	Positive or zero
0110	VS	V = 1	Overflow
0111	VC	V = 0	No overflow
1000	HI	C = 1 and Z = 0	Higher, unsigned
1001	LS	C = 0 or Z = 1	Lower or same, unsigned
1010	GE	N = V	Greater than or equal, signed
1011	LT	N != V	Less than, signed
1100	GT	Z = 0 and N = V	Greater than, signed
1101	LE	Z = 1 and N != V	Less than or equal, signed
1110	AL		Always.
1111	NV		Reserved (unused)

Code	Instruction
0000	AND
0001	EOR
0010	SUB
0011	RSB
0100	ADD
0101	ADC
0110	SBC
0111	RSC

Code	Instruction
1000	TST
1001	TEQ
1010	CMP
1011	CMN
1100	ORR
1101	MOV
1110	BIC
1111	MVN

Overall instruction format

- MOV instructions
  - `MOV, MVN`

`XXXCCS Rd, Rs , {shft #}`

31 28		24 21		19 16	15 12	11 7	6 5		3 0
Condition	000	Inst. type	S	0000	Rd	Shift size	shft	0	Rs

`XXXCCS Rd, Rs , {shft #}`

31 28		24 21		19 16	15 12	11	7	6 5		3 0
Condition	000	Inst. type	S	0000	Rd	Rshift	0	shft	1	Rs

`XXXCC Rd, #immediate (rotated)`

31 28		24 21		19 16	15 12	11 8	7 0
Condition	001	Inst. type	S	0000	Rd	#rot	8-bit immediate

Shift/Rotate instructions

- Instructions are encoded as MOV instructions

  E.g. LSL R1, R2, #5 ≡ MOV R1, R2, LSL #5

- Specify the shift type in bits 5 and 6

Code	Shift type
00	LSL
01	LSR
10	ASR
11	ROR/RRX

XXXCCS Rd, Rs , {shft #}

31–28	27–25	24–21	20	19–16	15–12	11–7	6–5	4	3–0
Condition	000	Inst. type	S	0000	Rd	Shift size	shft	0	Rs

XXXCCS Rd, Rs , {shft #}

31–28	27–25	24–21	20	19–16	15–12	11–8	7	6–5	4	3–0
Condition	000	Inst. type	S	0000	Rd	Rshift	0	shft	1	Rs

Q3 which of the following hexadecimal encoding of MOVLE R7,R4

1. 0xD0D07400
2. 0xD0D07004
3. 0XD1A07004
4. 0xD1A07400

XXXCCS Rd, Rs , {shft #}

31 28	24	21	19	16 15	12 11	7 6	5	3	0
Condition	000	Inst. type	S	0000	Rd	Shift size	shft	0	Rs

XXXCCS Rd, Rs , {shft #}

31 28	24	21	19	16 15	12 11	7	6	5	3	0
Condition	000	Inst. type	S	0000	Rd	Rshift	0	shft	1	Rs

XXXCC Rd, #immediate (rotated)

31 28	24	21	19	16 15	12 11	8 7	0
Condition	001	Inst. type	S	0000	Rd	#rot	8-bit immediate

Code	Condition	Flags	Meaning
0000	EQ	Z = 1	Equal
0001	NE	Z = 0	Not equal
0010	CS or HS	C = 1	Higher or same, unsigned
0011	CC or LO	C = 0	Lower, unsigned
0100	MI	N = 1	Negative
0101	PL	N = 0	Positive or zero
0110	VS	V = 1	Overflow
0111	VC	V = 0	No overflow
1000	HI	C = 1 and Z = 0	Higher, unsigned
1001	LS	C = 0 or Z = 1	Lower or same, unsigned
1010	GE	N = V	Greater than or equal, signed
1011	LT	N != V	Less than, signed
1100	GT	Z = 0 and N = V	Greater than, signed
1101	LE	Z = 1 and N != V	Less than or equal, signed
1110	AL		Always.
1111	NV		Reserved (unused)

Code	Instruction
0000	AND
0001	EOR
0010	SUB
0011	RSB
0100	ADD
0101	ADC
0110	SBC
0111	RSC

Code	Instruction
1000	TST
1001	TEQ
1010	CMP
1011	CMN
1100	ORR
1101	MOV
1110	BIC
1111	MVN

Q3 which of the following hexadecimal encoding of MOVLE R7,R4

1. 0xD0D07400
2. 0xD0D07004
3. 0XD1A07004
4. 0xD1A07400

XXXCCS Rd, Rs , {shft #}

31 28	24	21	19	16 15	12 11	7 6	5	3	0
Condition	000	Inst. type	S	0000	Rd	Shift size	shft	0	Rs

XXXCCS Rd, Rs , {shft #}

31 28	24	21	19	16 15	12 11	7	6	5	3	0
Condition	000	Inst. type	S	0000	Rd	Rshift	0	shft	1	Rs

XXXCC Rd, #immediate (rotated)

31 28	24	21	19	16 15	12 11	8 7	0
Condition	001	Inst. type	S	0000	Rd	#rot	8-bit immediate

Code	Condition	Flags	Meaning
0000	**EQ**	Z = 1	Equal
0001	**NE**	Z = 0	Not equal
0010	**CS or HS**	C = 1	Higher or same, unsigned
0011	**CC or LO**	C = 0	Lower, unsigned
0100	**MI**	N = 1	Negative
0101	**PL**	N = 0	Positive or zero
0110	**VS**	V = 1	Overflow
0111	**VC**	V = 0	No overflow
1000	**HI**	C = 1 and Z = 0	Higher, unsigned
1001	**LS**	C = 0 or Z = 1	Lower or same, unsigned
1010	**GE**	N = V	Greater than or equal, signed
1011	**LT**	N != V	Less than, signed
1100	**GT**	Z = 0 and N = V	Greater than, signed
1101	**LE**	Z = 1 and N != V	Less than or equal, signed
1110	**AL**		Always.
1111	**NV**		Reserved (unused)

Code	Instruction
0000	**AND**
0001	**EOR**
0010	**SUB**
0011	**RSB**
0100	**ADD**
0101	**ADC**
0110	**SBC**
0111	**RSC**

Code	Instruction
1000	**TST**
1001	**TEQ**
1010	**CMP**
1011	**CMN**
1100	**ORR**
1101	**MOV**
1110	**BIC**
1111	**MVN**

Branch Instructions

- For conditional branch and branch with link

B(L)CC Offset

31	28	23	0
Condition	101	L	Offset

- 26 bits offset shifted to the right by 2 (branching to word addresses) ➔ calculated by the assembler

- Branch range: ±32 MBytes

Load/Store Instructions

- Load/Store word bye

LDR/STR{B}CC Rs,[RD,offset pre or post]W

31–28	27–26	25	24–21	20	19–16	15–12	11–7	6–5	4	3–0
Condition	01	0	P U B W	L	Rs	Rd	Shift size	shft	0	Rs

31–28	27–26	25	24–21	20	19–16	15–12	11–0
Condition	01	1	P U B W	L	Rs	Rd	Immediate value

- L bit determines load (1) or store (0)
- P bit determines indexing pre (1) or post (0)
- U determines offset direction add (1) or subtract (0)offset
- B determines byte (1) or word (0) access
- W is for write back (!) of the offset

Load/Store Instructions

- Load/Store Multiple instructions

```
LDM/STMFXCC   RsW,{register list}
```

31 28		24 21		19 16	15 12 11 0
Condition	100	P U ^ W	L	Rs	Register list

- L bit determines load (1) or store (0)
- P bit determines indexing before (1) or after (
- U determines offset direction add (1) or subtr
- ^ is “don’t care”
- W is for write back (!) after operation

PU	Instruction
10	STMFD
01	LDMFD
11	STMFA
00	LDMFA

Load/Store Instructions

- Load/Store Multiple instructions

`STMFD SP!,{R0-R6}`

31 28	24	21		19 16	15															0
1110	100	10^1	0	1101 (SP)	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1
					PC	LR	SP	R12	R11	R10	R9	R8	R7	R6	R5	R4	R3	R2	R1	R0

- L bit determines load (1) or store <u>(0)</u>
- P bit determines indexing before <u>(1)</u> or after (0)
- U determines offset direction add (1) or subtract <u>(0)</u>
- ^ is “don’t care”
- W is for write back (!) after operation <u>(1)</u>

Q4 If the registers list field is encoded with 0X40F0, what register list this represents?

31–28	27–25	24–21	20	19–16	15–0
Condition	100	P U ^ W	L	Rs	Register list

1. {R4,R15}

2. {R4-R7,LR}

3. {R1,R8-R11}

4. Don't know!

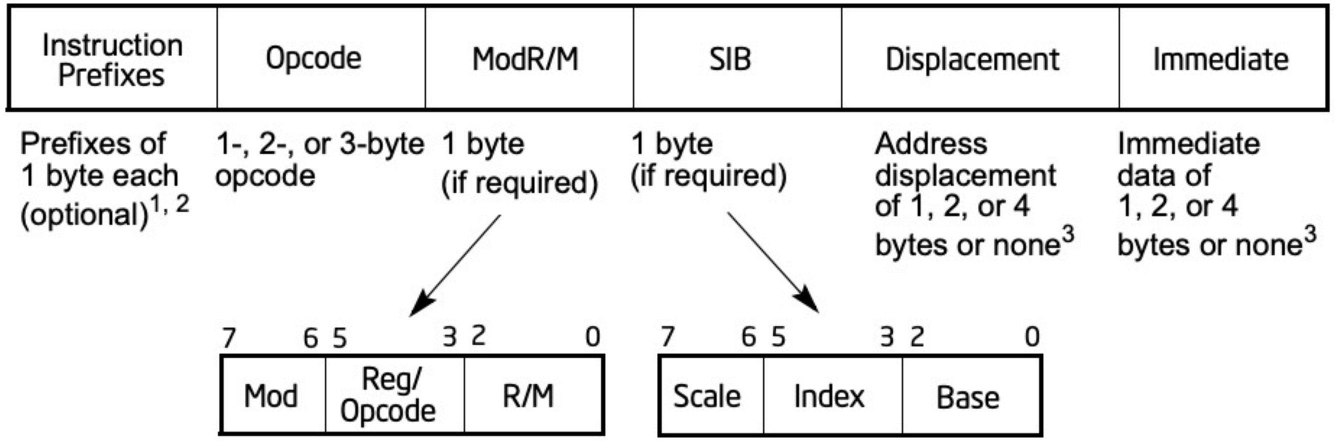
Answer is 2

Fixed vs. Variable-Length instruction

- ARM is fixed-length
  - Operands had to be registers or immediate values
- Variable length instructions
  - Intel 80x86: Instructions vary from 1 to 17 bytes long.
  - Digital VAX: Instructions vary from 1 to 54 bytes long.
  - Require multi-step fetch and decode.
  - Allow for a more flexible (but complex) and compact instruction set.

Variable-Length: Intel Instruction Format

- Complex instructions = complex hardware
- Higher variety for smaller code and faster execution

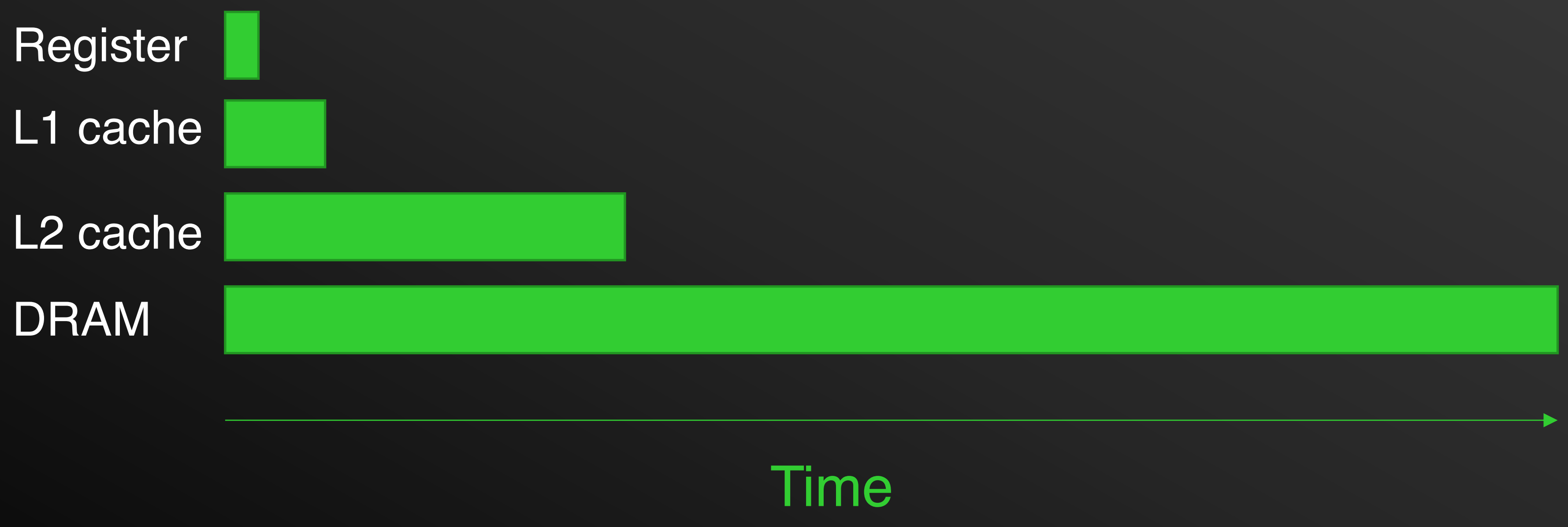


1. The REX prefix is optional, but if used must be immediately before the opcode; see Section 2.2.1, "REX Prefixes" for additional information.
2. For VEX encoding information, see Section 2.3, "Intel® Advanced Vector Extensions (Intel® AVX)".
3. Some rare instructions can take an 8B immediate or 8B displacement.

Figure 2-1. Intel 64 and IA-32 Architectures Instruction Format

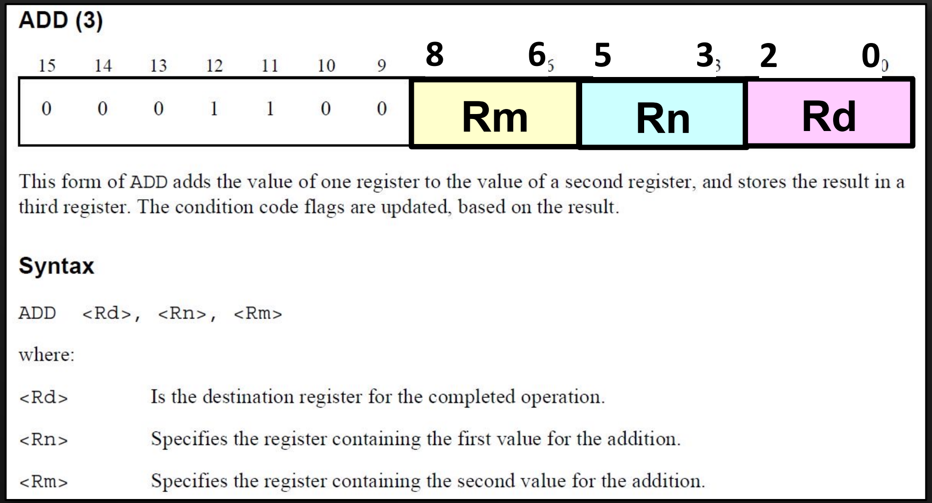
Using Registers

- Operations using registers are **faster** than those involving memory.
  - Data transfer between registers and ALU is faster due to it physically proximity.
  - To speed up code execution, keep as much of your computations within **registers**.



Implication of Register Count

- Implementing many registers within the CPU will incurs increasing overheads.
    - They take up precious **space** in the silicon die.
    - Connections to various busses increase the **routing** and **multiplexing complexity**.
    - More registers increases the **operand size** during instruction encoding.



ADD (3)

15	14	13	12	11	10	9	8 – 6	5 – 3	2 – 0
0	0	0	1	1	0	0	Rm	Rn	Rd

This form of ADD adds the value of one register to the value of a second register, and stores the result in a third register. The condition code flags are updated, based on the result.

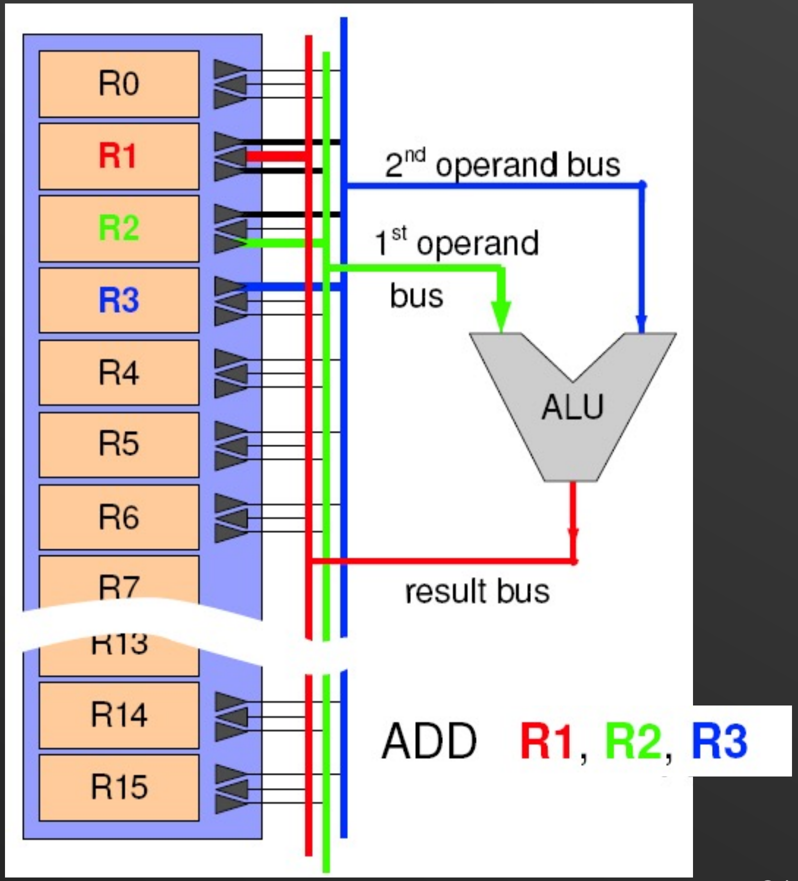
Syntax

`ADD <Rd>, <Rn>, <Rm>`

where:

<Rd>	Is the destination register for the completed operation.
<Rn>	Specifies the register containing the first value for the addition.
<Rm>	Specifies the register containing the second value for the addition.

Instruction Encoding for ADD in Thumb-2



Orthogonality of ISA

- In a truly orthogonal ISA, every instruction is able to use every available addressing modes.
- A truly orthogonal ISA does not restrict certain instruction from using only specific registers.
- Difficult to achieve complete orthogonality due to the **large number of bit patterns** required to express all combinations of operations and addressing modes.
- Increase instruction length will increases the memory size required by the program.

Summary - Why is Good Instruction Set Architecture (ISA) Design Important?

- Good ISA design can yield benefits such as:
  - Increases the execution performance of the processor.
  - Increases the code density (i.e. how much memory a piece of code will occupy).
  - Reduces the power consumption of the CPU.
  - Reduces manufacturing cost of processor.
  - Easy for programmers to write efficient programs.
  - Efficient mapping of high-level programming requirements into low-level instruction sets.

Why Should I Care About Processor Designs and ISA?

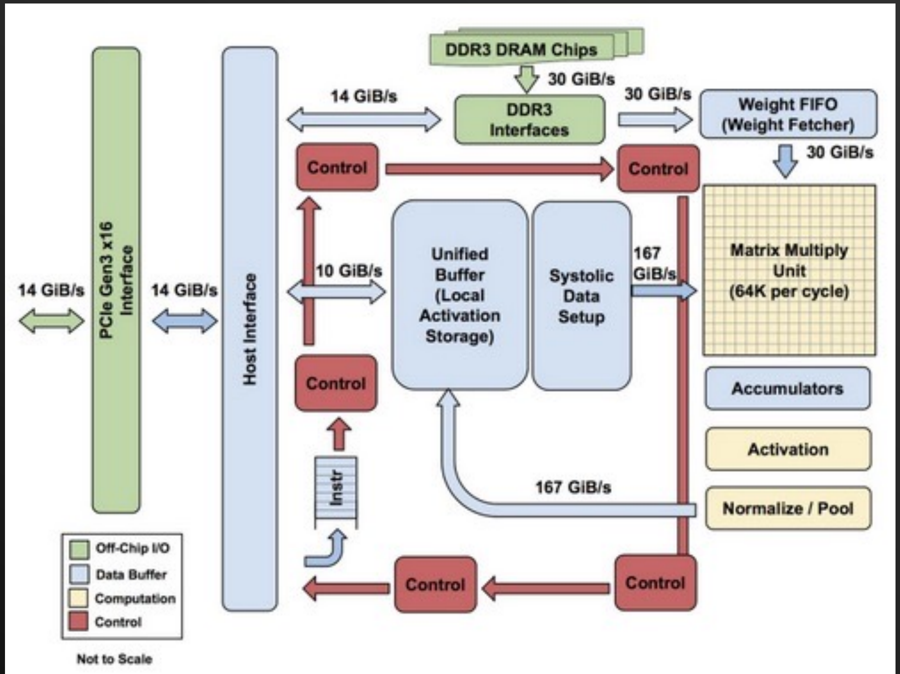
- Designing a processor (or a co-processor) is no longer a hardware company job

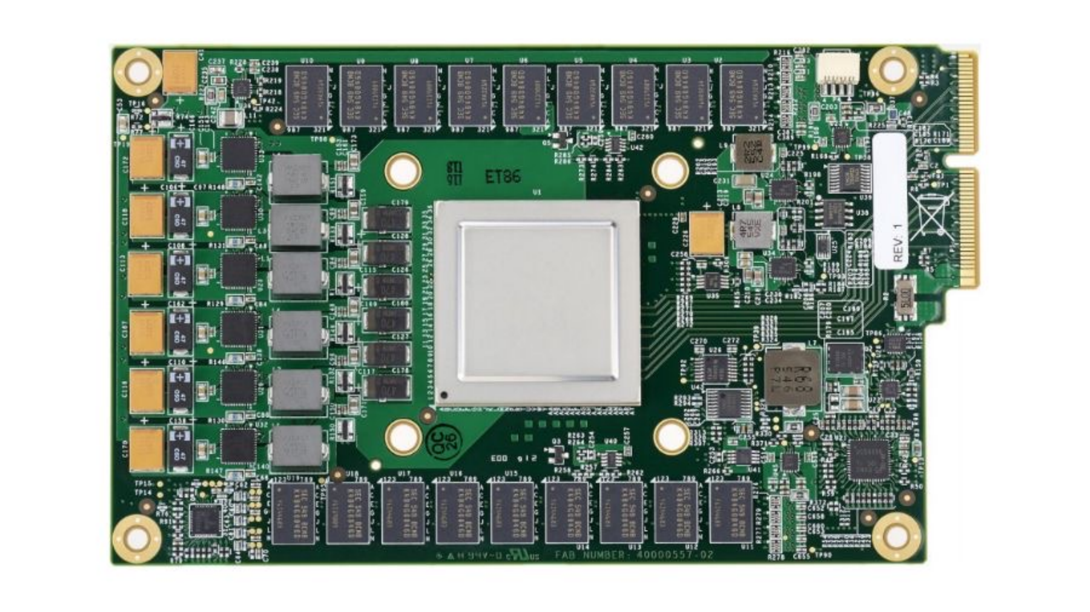
Why Should I Care About Processor Designs and ISA?

- Designing a processor (or a co-processor) is no longer a hardware company job

The Google TPU

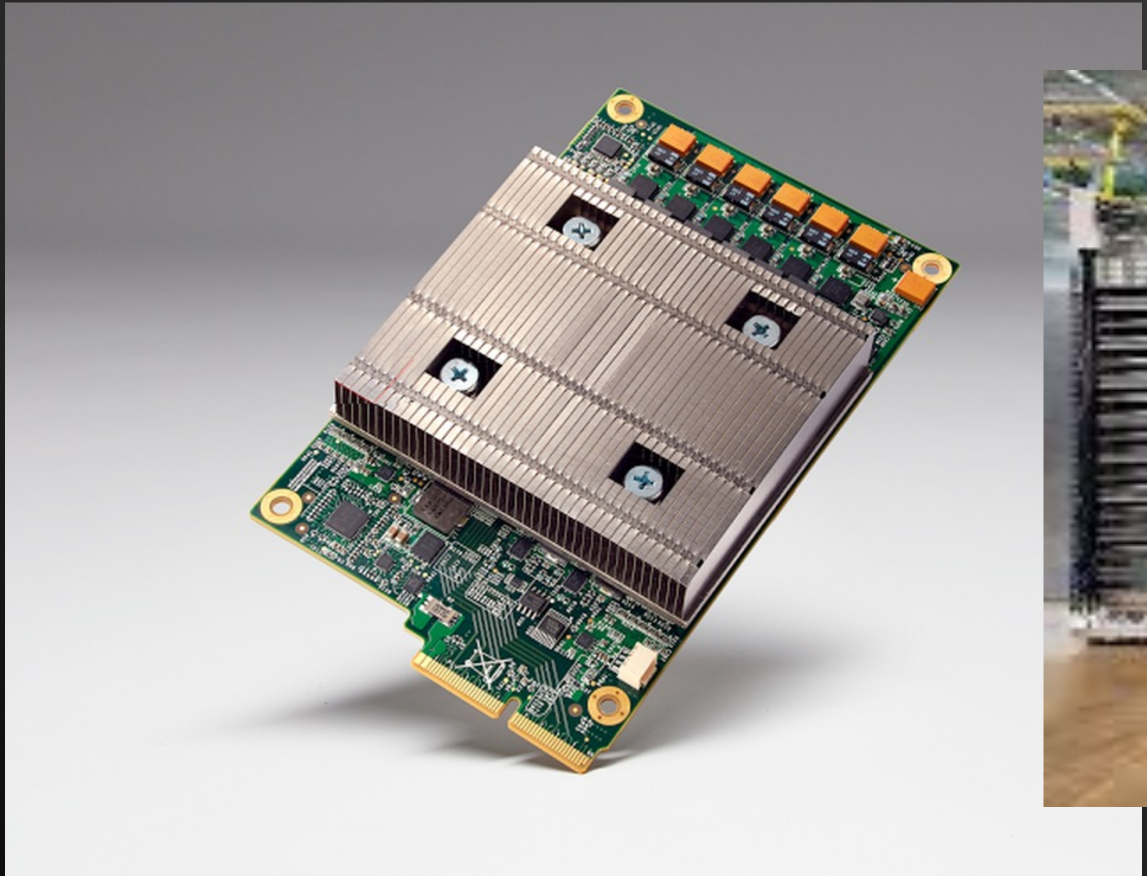
- Tensor Processing Unit
- Designed by Engineers in Google
- Tailored to make AI applications run faster





The Google TPU

- Design by Engineers in Google to make AI applications run faster



Summary

- ARM is fixed-length load/store architecture
  - Operands reside in registers
- Other architectures can use variable-length format
  - E.g, Intel architecture
- Use of registers is key towards higher performance